Loose coupling using interface : [SOLID [Dense Coding Hardline Polymorphism]]

Loose Coupling: If the **degree of dependence** from one class object to another class is **very low** then it is called loose coupling. Interface is required!

Tightly coupled : If the degree of dependency of one class to another class is very **high** then it is called Tightly coupled.

According to IT industry standard, we should always prefer loose coupling so the maintenance of the project will become easy.

High Cohesion (Encapsulation)

Our private data must be accessible via public methods (setter and getters) so, in between data and method we must have high cohesion
(high coupling so, validation of outer data is possible.

[Program on loose coupling]

```
package com.rest.loose_coupling;

public interface RawMaterial
{
    void prepare();
}

package com.rest.loose_coupling;

public class Tea implements RawMaterial
{
    public void prepare()
    {
        Sp.println("Preparing [redacted] Tea");
    }
}

package com.rest.loose_coupling;

public class Coffee implements RawMaterial
{
    @Override
    public void prepare()
    {
        Sp.println("Preparing Java Coffee");
    }
}

package com.rest.loose_coupling;

public class Horlicks implements RawDrink
{
    @Override
    public void prepare()
    {
        Sp.println("Preparing Horlicks");
    }
}

package com.rest.loose_coupling;

public class Restaurant
{
    public static void prepareDrink(RawMaterial rm) /loose coupling
    {
        rm.prepare();
    }
}

package com.rest.loose_coupling;

public class LooseCoupling {
    public static void main(String[] args)
    {
        Restaurant.prepareDrink(new Tea());
        Restaurant.prepareDrink(new Coffee());
        Restaurant.prepareDrink(new Horlicks());
    }
}
```

Method return type as an interface :

* Whenever we take method return type as a interface then it is called **Factory Design Pattern.**

Example :

```
public RawDrink method()
{
    return null;
}
```

Diagram

MethOne (Method return type)

new Tea(); new Coffee(); new Horlicks(); (return value)

SOP to show the interface as a method return type with anonymous inner class using reference :

```
package com.rest.factory_design_pattern;

public class FactoryDesignWithReference
{
    public static void main(String[] args)
    {
        Printable p1 = Printer.getPrinter();
        p1.print();
    }
}

interface Printable
{
    void print();
}

class Printer
{
    public static Printable getPrinter()
    {
        Printable p1 = new Printable();

        p1.
        {
            @Override
            public void print();
            {
                Sp.println("Printing Through Anonymous Inner class");
            };
        };

        return p1;
    }
}
```

Very very important program for JDBC and Spring Framework to understand Dependency Injection.

SOP to show the interface as a method return type with anonymous inner class without using reference

```
package com.rest.factory_design_pattern;

public class FactoryDesignWithoutRef
{
    public static void main(String[] args)
    {
        Printable p = Printer.getPrinter();
        p.print();
    }
}

interface Printable
{
    void print();
}

class Printer
{
    public static Printable getPrinter()
    {
        return new Printable()
        {
            @Override
            public void print()
            {
                Sp.println("SOP without ref");
            }
        };
    }
}
```

How to achieve multiple inheritance using interface :

```
interface A
{
    void m1();
}

                interface B
                {
                    void m1();
                }

                    class C implements A,B
                    {
                        @1
                        @2
                        void m1()
                        {
                        }
                    }

                                public class Object
                                {
                                    public Object()
                                    {
                                    }
                                }
```

In a class we have a constructor so, it is annoying ambiguity issue but inside an interface we don't have constructor so multiple inheritance is possible using interface as shown in the program below.

The Implementer class constructor's super keyword will directly move to Object class constructor (shown in the above diagram)

```
package com.rest.m1;

interface A
{
    void m1();
}
interface B
{
    void m1();
}
class Implementer implements A,B
{
    public void m1()
    {
        Sp.println("M1 is possible in Java using interface");
    }
}

public class MultipleInheritance {

    public static void main(String[] args)
    {
        Implementer I = new Implementer();
        I.m1();
    }
}
```

Extending an interface :

* An interface can extend into another interface. It cannot extend a class.
  [If an interface will extend a class then all the concrete method will come inside interface]

```
package com.rest.InheriteFrom_Impl;

interface A
{
    void m1();
}
interface B extends A
{
    void m1();
}

class MyClass implements B
{
    @Override
    public void m1()
    {
        Sp.println("m1 method overrided");
    }

    @Override
    public void m1()
    {
        Sp.println("m1 method overrided");
    }
}

public class ExtendingInterface {
    public static void main(String[] args)
    {
        MyClass myClass = new MyClass();
        myClass.m1();
        myClass.m1();
    }
}
```